PRACTICAL-7

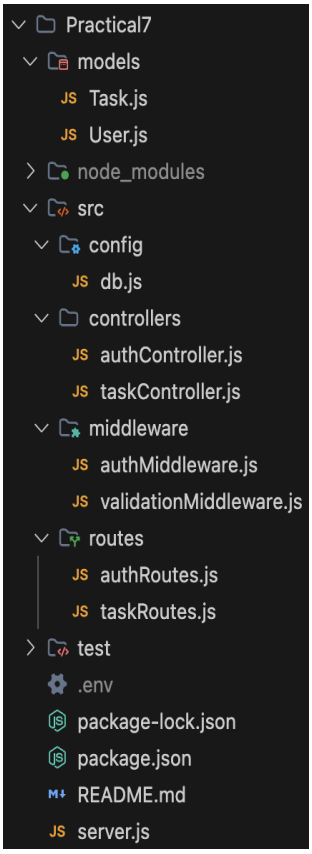
PROBLEM DEFINITION:

• Add user registration and login functionality to the Task Management application.
• Hash passwords using bcrypt before saving the User document to MongoDB.
• Implement JWT token generation on successful login, with a reasonable expiry (e.g., 1 hour).
• Protect all task routes using an authentication middleware that verifies the JWT.
• Add server-side input validation that rejects malformed requests (e.g., missing title) before they reach the database.

SUPPLEMENTARY PROBLEMS:

• Add a /me endpoint that returns the currently logged-in user's details using the decoded JWT.
• Implement token expiry handling on the frontend redirect to login if a protected request returns 401.
• Add a logout mechanism that clears the token from the client.

ARCHITECTURE / DIAGRAM:

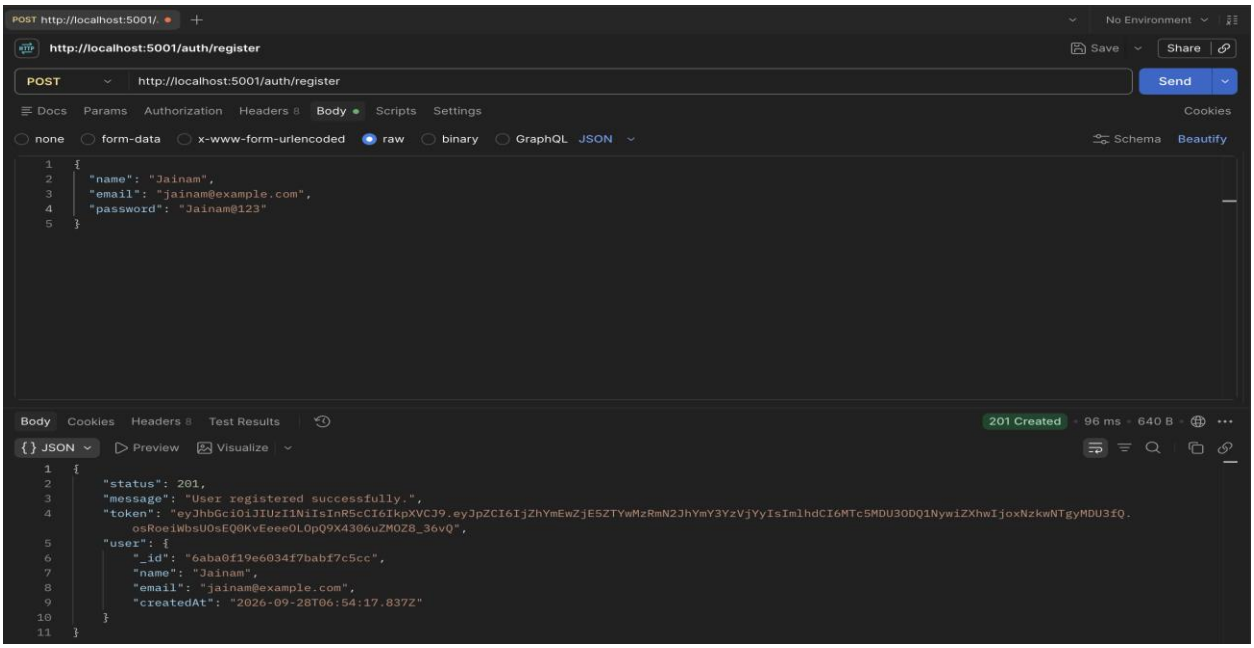


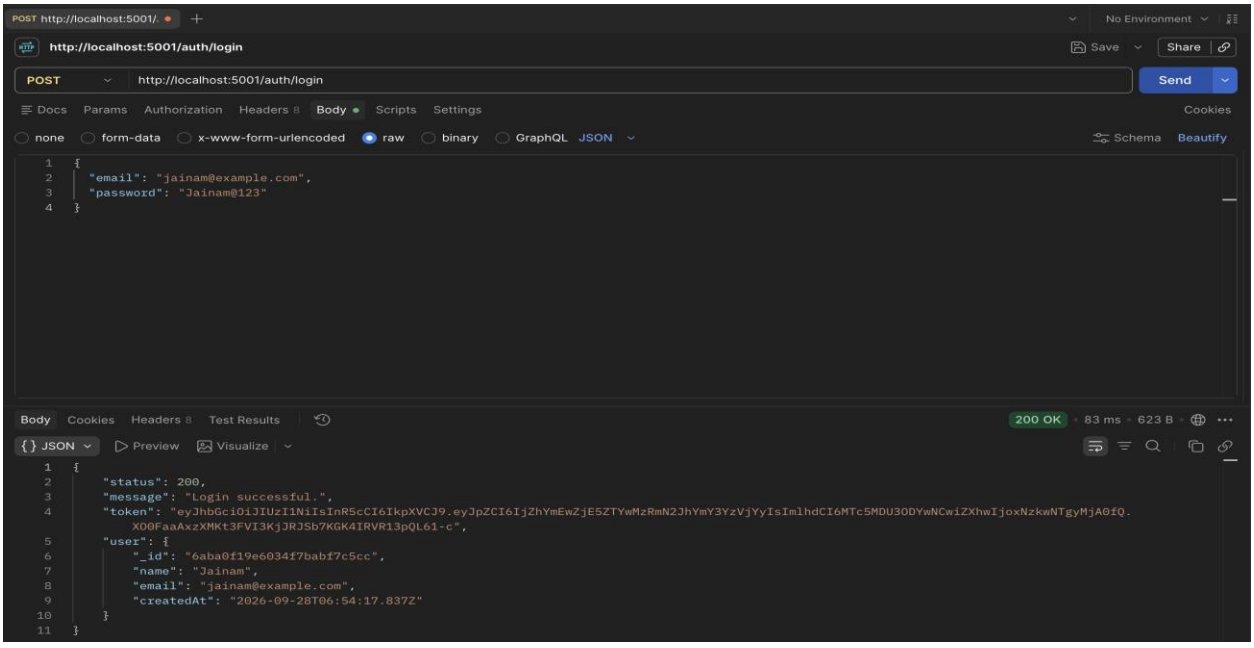
OUTPUT (SUPPLEMENTARY PROBLEMS INCLUDED):

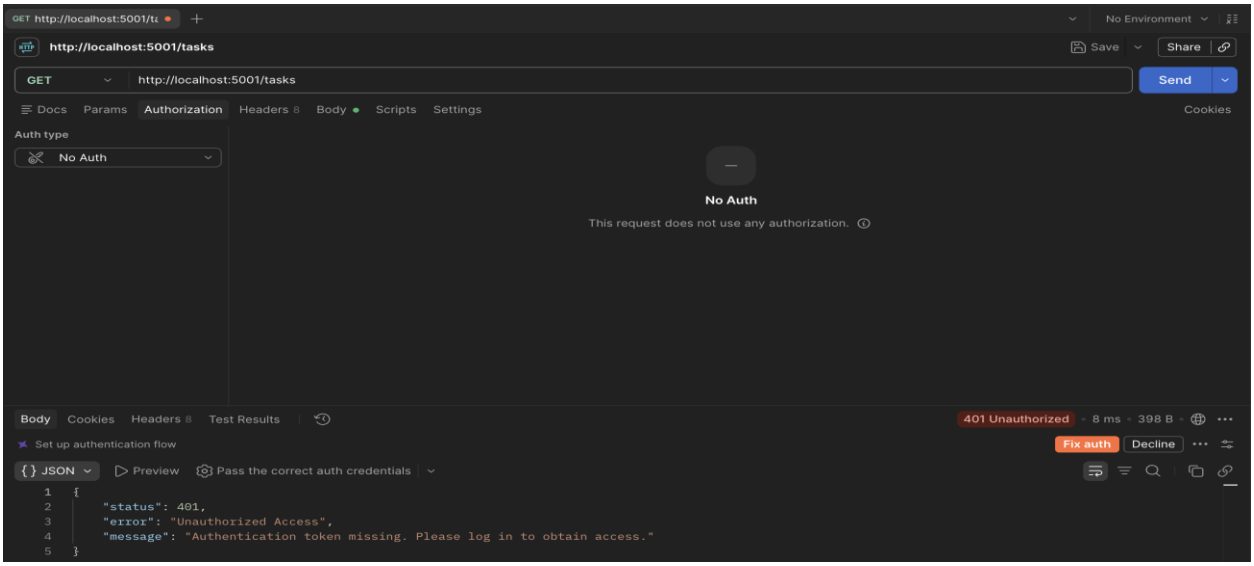
```
kamanijainamrajeshkumar@Kamanis-MacBook-Air Practicals % cd Practical7
kamanijainamrajeshkumar@Kamanis-MacBook-Air Practical7 % npm start

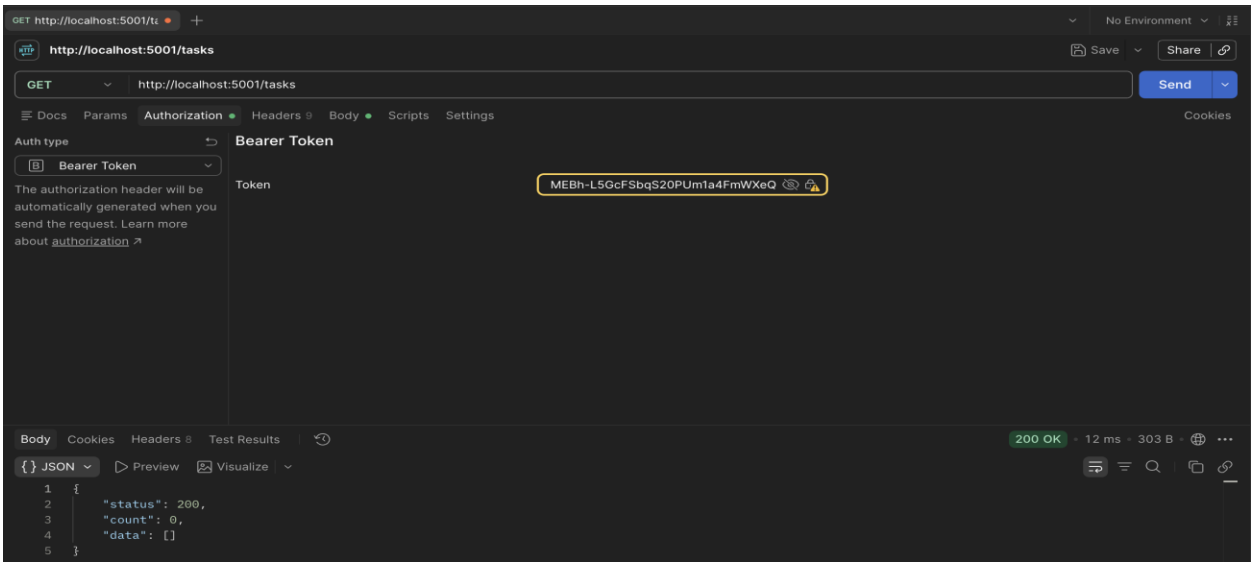
> practical7-jwt-auth-task-api@1.0.0 start
> node server.js

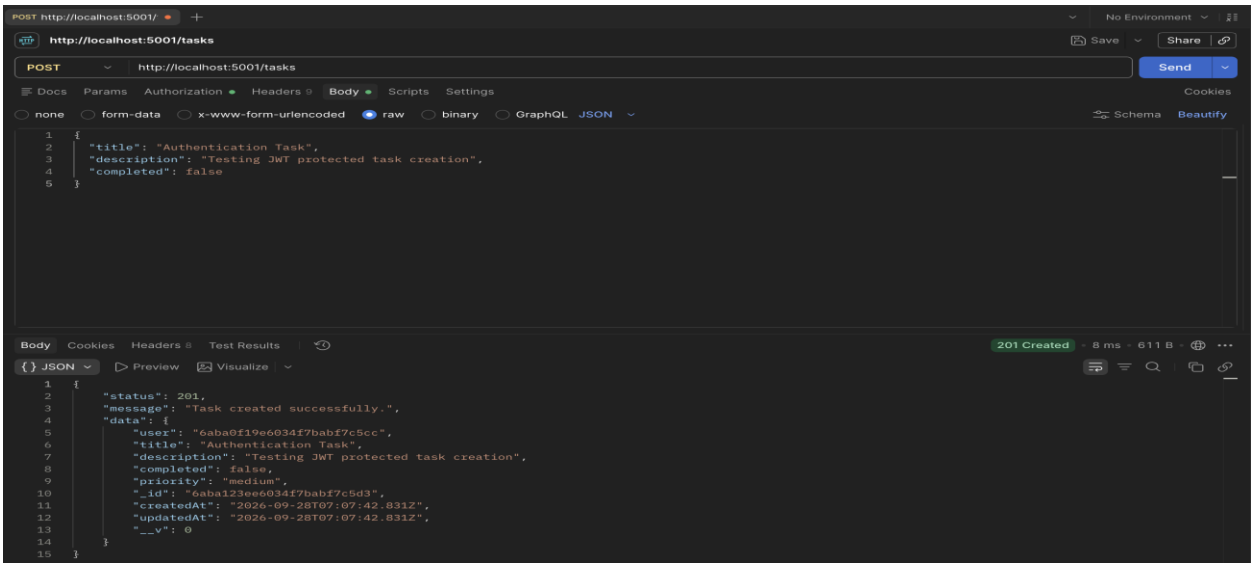
⚡  Connected to In-Memory MongoDB Server: mongodb://127.0.0.1:29650/
============================================================
🚀 Practical 7: JWT Auth & Task API running on port 5001
🔑 Auth Endpoint: http://localhost:5001/auth
🔒 Task Endpoint: http://localhost:5001/tasks
============================================================
```

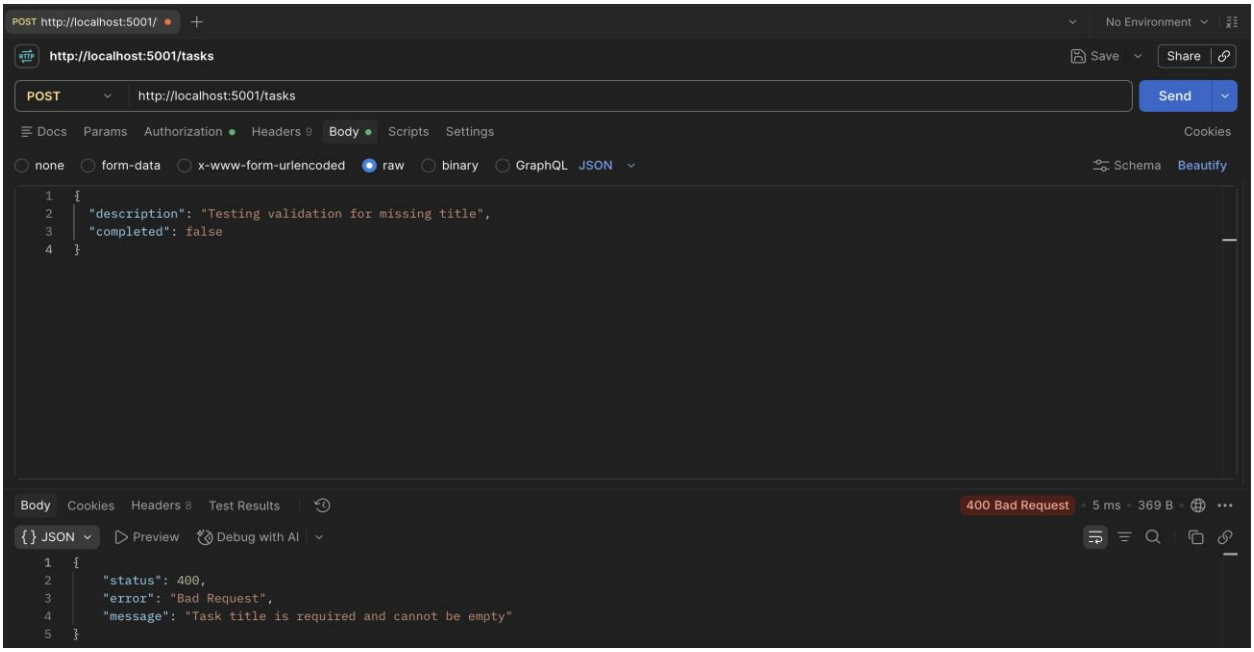


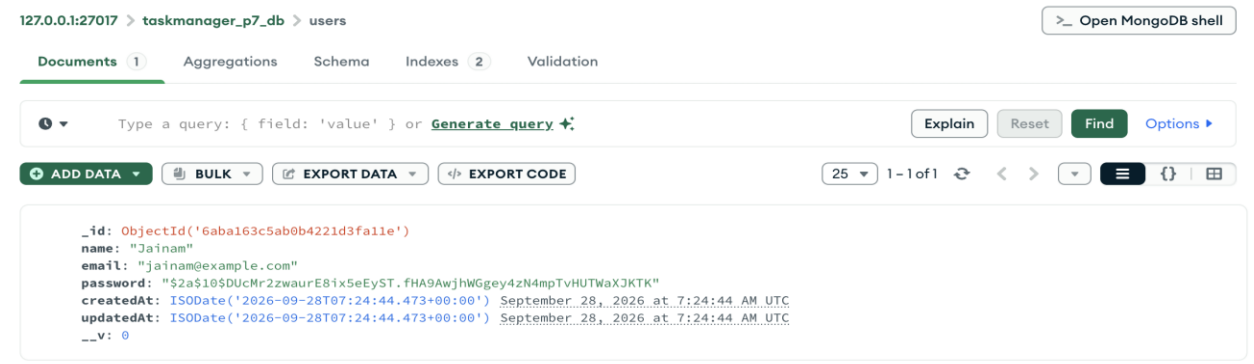


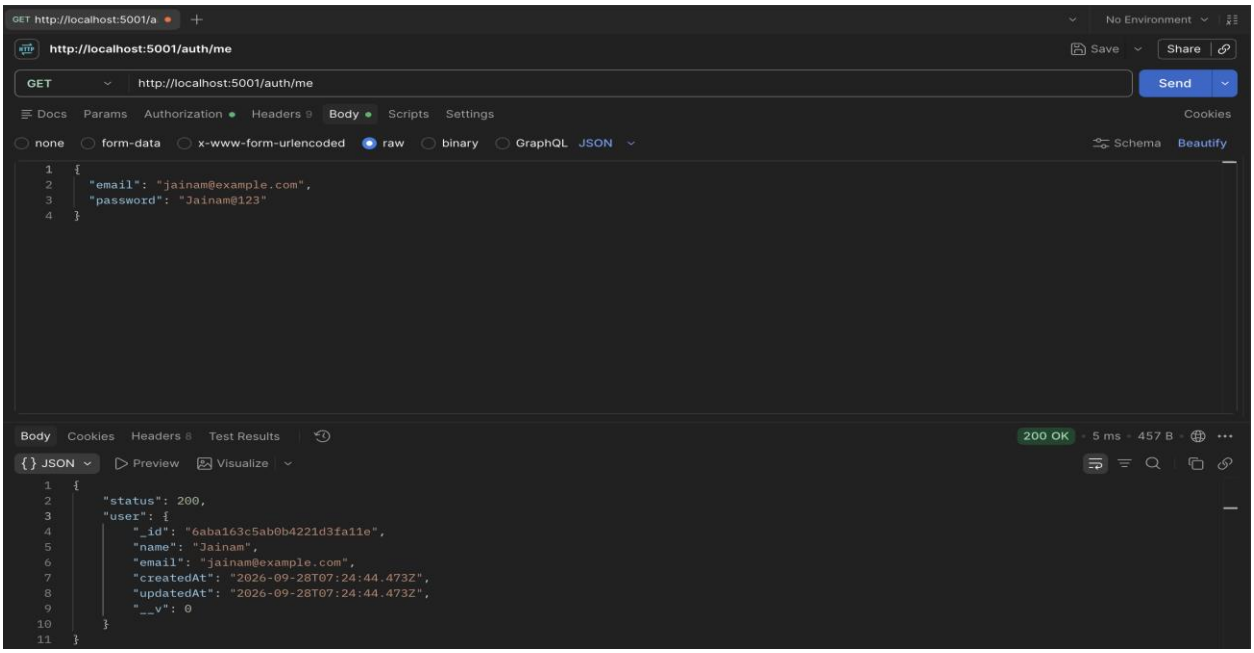

GET http://localhost:5001/ta
No Environment
http://localhost:5001/tasks
Save
Share
GET
http://localhost:5001/tasks
Send
Docs Params Authorization Headers 8 Body Scripts Settings
Cookies
Auth type
No Auth
No Auth
This request does not use any authorization.
Body Cookies Headers 8 Test Results
401 Unauthorized 8 ms 398 B
Set up authentication flow
Fix auth Decline
{ } JSON Preview Pass the correct auth credentials
{
"status": 401,
"error": "Unauthorized Access",
"message": "Authentication token missing. Please log in to obtain access."
}

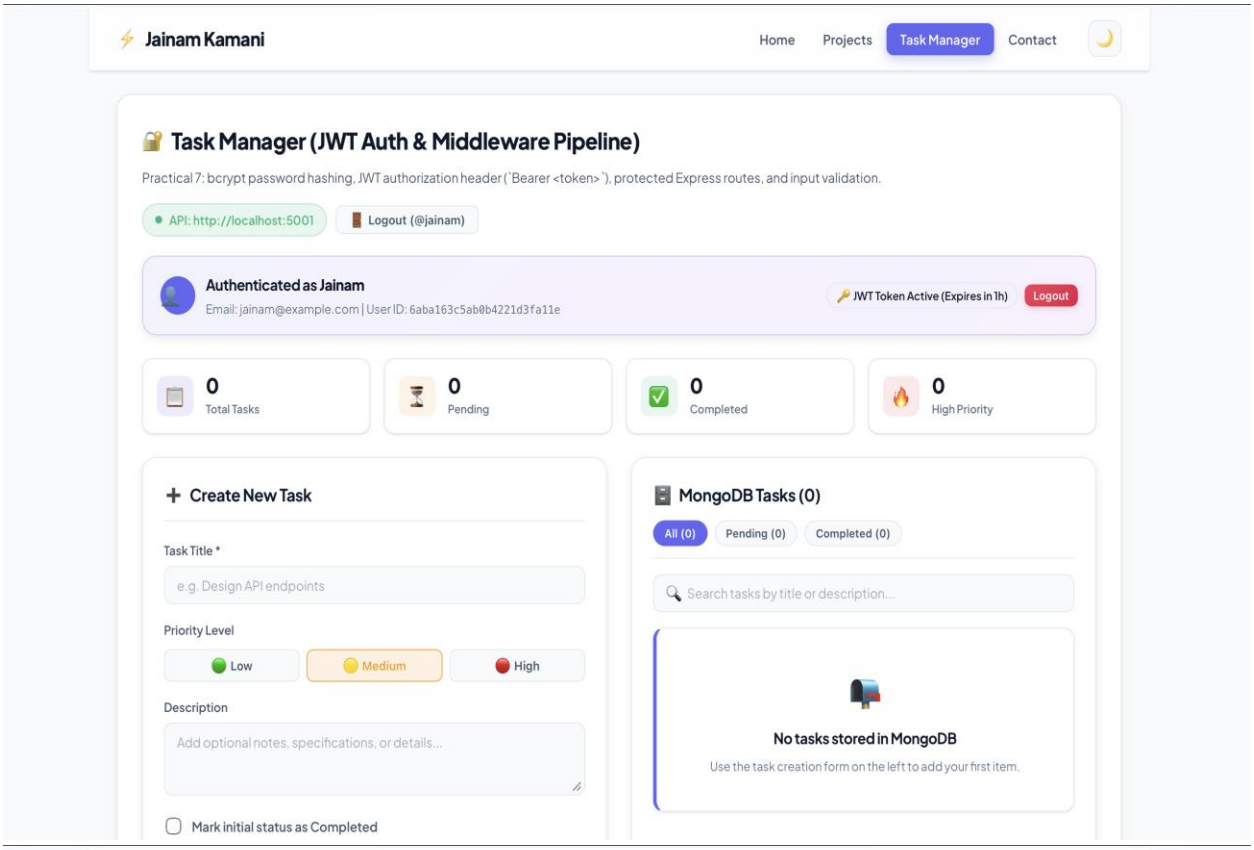

GET http://localhost:5001/ta
No Environment
http://localhost:5001/tasks
Save
Share
GET
http://localhost:5001/tasks
Send
Docs Params Authorization Headers 9 Body Scripts Settings
Cookies
Auth type
Bearer Token
Bearer Token
Token
MEBh-L5GcFSbqS20PUm1a4FmWXeQ
The authorization header will be automatically generated when you send the request. Learn more about authorization
Body Cookies Headers 8 Test Results
200 OK 12 ms 303 B
{ } JSON Preview Visualize
{
"status": 200,
"count": 0,
"data": []
}

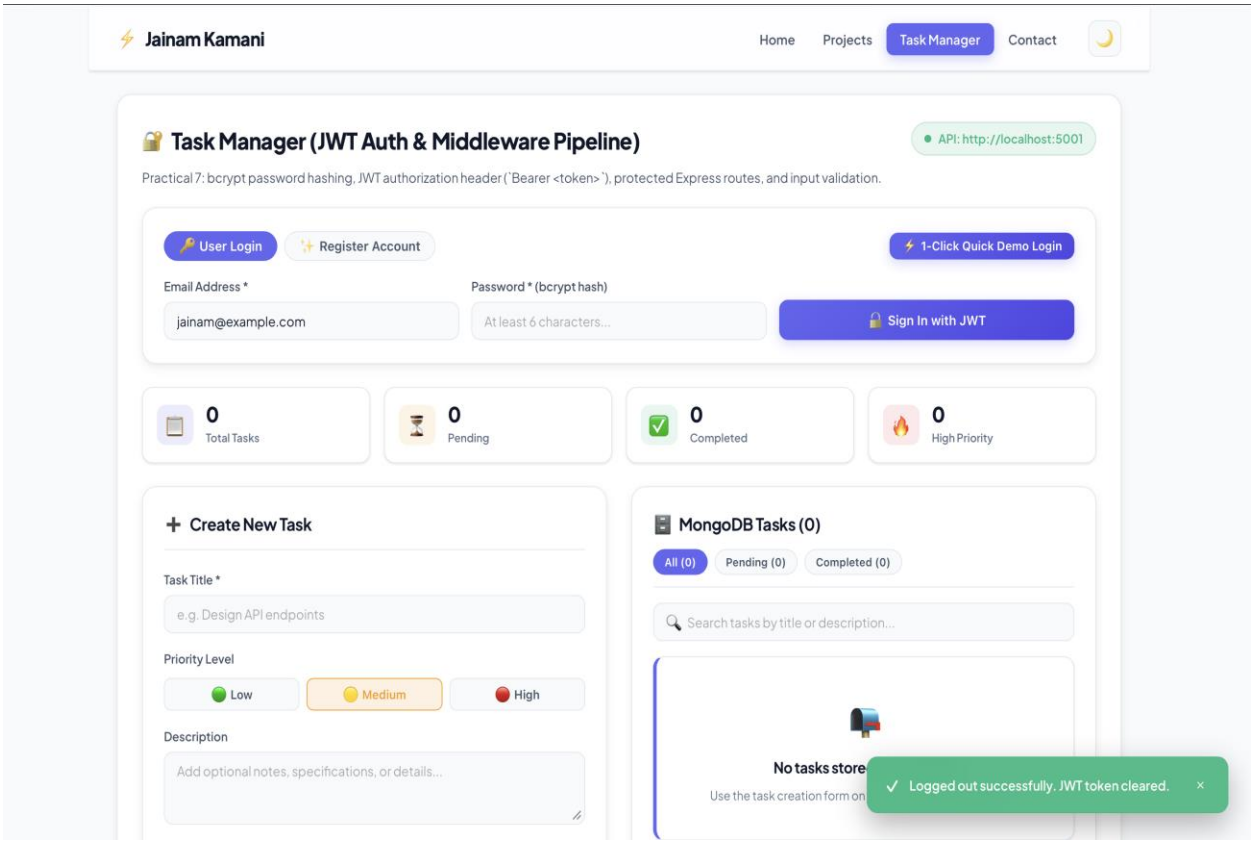

POST http://localhost:5001/
No Environment
http://localhost:5001/tasks
Save
Share
POST
http://localhost:5001/tasks
Send
Docs Params Authorization Headers 9 Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"title": "Authentication Task",
"description": "Testing JWT protected task creation",
"completed": false
}
Body Cookies Headers 8 Test Results
201 Created 8 ms 611 B
{ } JSON Preview Visualize
{
"status": 201,
"message": "Task created successfully.",
"data": {
"user": "6aba0f19e6034f7babf7c5cc",
"title": "Authentication Task",
"description": "Testing JWT protected task creation",
"completed": false,
"priority": "medium",
"_id": "6aba123ee6034f7babf7c5d3",
"createdAt": "2026-09-28T07:07:42.831Z",
"updatedAt": "2026-09-28T07:07:42.831Z",
"__v": 0
}
}


POST http://localhost:5001/
No Environment
http://localhost:5001/tasks
Save
Share
POST
http://localhost:5001/tasks
Send
Docs Params Authorization Headers 9 Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"description": "Testing validation for missing title",
"completed": false
}
Body Cookies Headers 8 Test Results
400 Bad Request 5 ms 369 B
{} JSON Preview Debug with AI
{
"status": 400,
"error": "Bad Request",
"message": "Task title is required and cannot be empty"
}


127.0.0.1:27017 > taskmanager_p7_db > users
Open MongoDB shell
Documents 1 Aggregations Schema Indexes 2 Validation
Type a query: { field: 'value' } or Generate query
Explain Reset Find Options
ADD DATA BULK EXPORT DATA EXPORT CODE
25 1 - 1 of 1
_id: ObjectId('6aba163c5ab0b4221d3fa11e')
name: "Jainam"
email: "jainam@example.com"
password: "$2a$10$DUcMr2zwaurE8ix5eEyST.fHA9AwjhWGgey4zN4mpTvHUTWaXJKTK"
createdAt: ISODate('2026-09-28T07:24:44.473+00:00') September 28, 2026 at 7:24:44 AM UTC
updatedAt: ISODate('2026-09-28T07:24:44.473+00:00') September 28, 2026 at 7:24:44 AM UTC
__v: 0


GET http://localhost:5001/a
No Environment
http://localhost:5001/auth/me
Save
Share
GET
http://localhost:5001/auth/me
Send
Docs Params Authorization Headers 9 Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"email": "jainam@example.com",
"password": "Jainam@123"
}
Body Cookies Headers 8 Test Results
200 OK 5 ms 457 B
{} JSON Preview Visualize
{
"status": 200,
"user": {
"_id": "6aba163c5ab0b4221d3fa11e",
"name": "Jainam",
"email": "jainam@example.com",
"createdAt": "2026-09-28T07:24:44.473Z",
"updatedAt": "2026-09-28T07:24:44.473Z",
"__v": 0
}
}


Jainam Kamani
Home
Projects
Task Manager
Contact
Task Manager (JWT Auth & Middleware Pipeline)
Practical 7: bcrypt password hashing, JWT authorization header (`Bearer <token>`), protected Express routes, and input validation.
API: http://localhost:5001
Logout (@jainam)
Authenticated as Jainam
Email: jainam@example.com | User ID: 6aba163c5ab0b4221d3fa11e
JWT Token Active (Expires in 1h)
Logout
0
Total Tasks
0
Pending
0
Completed
0
High Priority
Create New Task
Task Title *
e.g. Design API endpoints
Priority Level
Low
Medium
High
Description
Add optional notes, specifications, or details...
Mark initial status as Completed
MongoDB Tasks (0)
All (0)
Pending (0)
Completed (0)
Search tasks by title or description...
No tasks stored in MongoDB
Use the task creation form on the left to add your first item.


Jainam Kamani
Home
Projects
Task Manager
Contact
Task Manager (JWT Auth & Middleware Pipeline)
API: http://localhost:5001
Practical 7: bcrypt password hashing, JWT authorization header (`Bearer <token>`), protected Express routes, and input validation.
User Login
Register Account
1-Click Quick Demo Login
Email Address *
jainam@example.com
Password * (bcrypt hash)
At least 6 characters...
Sign In with JWT
0
Total Tasks
0
Pending
0
Completed
0
High Priority
Create New Task
Task Title *
e.g. Design API endpoints
Priority Level
Low
Medium
High
Description
Add optional notes, specifications, or details...
MongoDB Tasks (0)
All (0)
Pending (0)
Completed (0)
Search tasks by title or description...
Logged out successfully. JWT token cleared.

KEY QUESTIONS / ANALYSIS:

1) **Why must passwords be hashed before storage instead of saved as plain text, even in a lab/demo project?**

Passwords must be hashed using a library such as bcrypt before storing them in MongoDB. Hashing protects users' passwords if the database is exposed and prevents attackers from reading the original passwords directly. It follows essential security practices, even in a lab project.

2) **What does the authentication middleware actually verify, and what happens if the token is missing or expired?**

The authentication middleware verifies the JWT's signature and expiry to ensure that the token is valid. If the token is missing, invalid, or expired, the middleware rejects the request with a 401 Unauthorized response. If valid, it identifies the user and attaches the user information to req.user.

3) **Why should input validation happen on the server even if the frontend already validates the same fields?**

Server-side validation is necessary because frontend validation can be bypassed by sending requests directly through tools such as Postman. The backend must validate incoming data to reject missing, malformed, or invalid values before they reach the database, ensuring data integrity and security.

GITHUB REPOSITORY:

https://github.com/9081495207/ITUE301-Advanced-Web-Development-Frameworks